

SUMMARY

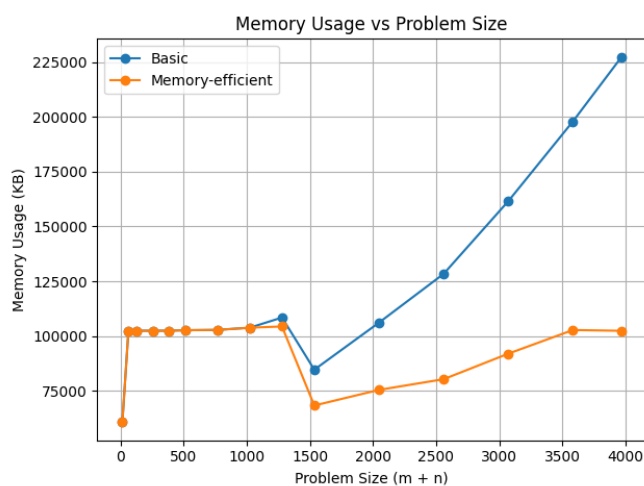
USC ID/s: 4534167714, 3888315499, 6300850270, 1214333928, 3038472455

Datapoints

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.438	0.089	60752	60768
64	1.809	0.878	102384	102384
128	1.847	2.981	102416	102416
256	5.871	10.554	102432	102432
384	13.406	24.468	102432	102432
512	23.448	42.795	102656	102656
768	56.978	99.601	102800	102800
1024	101.145	172.909	103792	103808
1280	162.608	271.665	108432	104368
1536	228.236	357.424	84640	68304
2048	430.541	676.470	106112	75424
2560	676.031	1070.075	128368	80304
3072	934.767	1473.812	161536	91952
3584	1313.007	2085.271	197952	102752
3968	1632.035	2635.740	227280	102384

Insights

Graph1 – Memory vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial

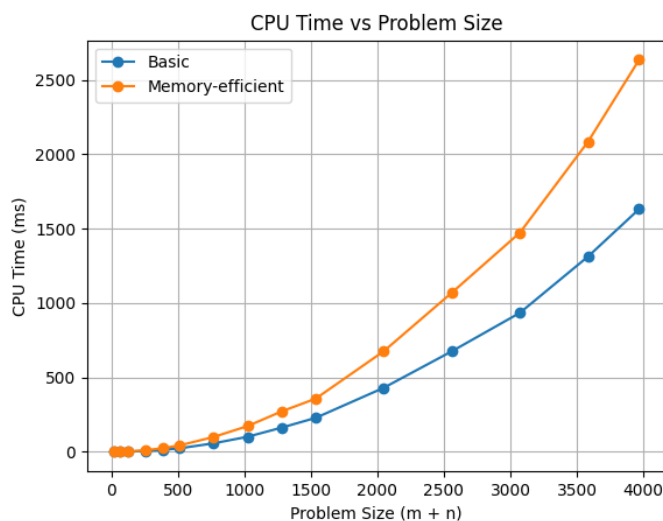
Efficient: Linear

Explanation:

The blue line representing the memory usage of the basic function is consistently above the orange line representing the memory usage of the efficient function. This makes sense, since the efficient function is optimized to reduce memory usage, using only $O(m + n)$ space. The basic algorithm uses $O(mn)$ space, as seen by the blue line's much steeper, accelerating growth.

Note: memory appears high and flat before problem size is 1500 because Python's baseline memory overhead dominates small inputs. The drop near 1500 is because Python's memory allocator reuses and releases memory in chunks, causing temporary decreases even though overall memory usage increases with input size.

Graph2 – Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial

Efficient: Polynomial

Explanation:

Both the basic and memory efficient solutions have a runtime of $O(mn)$. Since m and n grow proportionally, $O(mn)$ in this graph behaves like $O((m+n)^2)$, which explains the upward-curving (quadratic-like) shape of both lines. The memory-efficient algorithm has additional overhead from the divide-and-conquer recursion, making it slower in practice despite matching the basic algorithm's asymptotic time complexity. As a result, the memory efficient solution (orange line) uses more time than the basic algorithm (blue line).

Optimal Trivial Size

The optimal trivial size for our algorithm to perform optimally is a problem size ($m+n$) of about 1000. Looking at the memory graph, up until problem size = 1000, the two algorithms use roughly the same amount of memory, since they are dominated by Python's baseline memory overhead. In terms of runtime, the basic algorithm consistently has a faster runtime than the efficient solution because it avoids the overhead of repeated recursive splits and split-cost computations.

After 1000, the memory-efficient algorithm begins to significantly outperform the basic algorithm in terms of space usage, as the basic algorithm's $O(mn)$ memory grows rapidly. But for problem sizes ≤ 1000 , the basic algorithm suffices. Therefore, 1000 is the optimal trivial size that keeps the fast runtime of the basic solution while minimizing the extra space overhead of basic compared to efficient.

Contribution

(Please mention what each member did if you think everyone in the group does not have an equal contribution, otherwise, write "Equal Contribution")

<USC ID/s>: <Equal Contribution>

4534167714, 3888315499, 6300850270, 1214333928, 3038472455: Equal Contribution